

Capstone_Report_Team_6

by Chetan Nadichagi

Submission date: 21-Apr-2026 12:03PM (UTC+0530)

Submission ID: 2939045724

File name: Capstone_Report_Team_8.pdf (1.69M)

Word count: 5096

Character count: 27887

ABSTRACT

Driver fatigue and distractions are considered the leading causes of traffic collisions worldwide resulting in significant loss of human and economic capital. Identifying driver behavior in advance is crucial to improving road safety through accident prevention. Herein, a real-time driver behavior anomaly detection model leveraging deep learning techniques is proposed.

The driver behavior detection pipeline is based on extracting facial features using the MediaPipe Face Mesh library followed by the analysis of the features for physiological indicators including eye blinks, eye closure, and yawning. Also, YOLOv8 neural networks are employed to detect driver's objects of attention in order to identify potential distracting factors including usage of a mobile phone while driving.

The combination of MediaPipe Face Mesh for physiological detection and YOLOv8 for distraction identification increases detection accuracy while reducing false alarms. The proposed solution relies on temporal persistence which means that behavior is analyzed over several frames rather than one. Moreover, the proposed detection algorithm runs on the standard laptop webcam only, making it compact and inexpensive.

In terms of performance, our detection algorithm is able to efficiently identify driver anomalies and send corresponding notifications. Our approach may be considered a promising way to develop efficient intelligent driver monitoring solutions.

1 TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	10
2.	PROBLEM DEFINITION	11
3.	DATA	
3.1	Overview	12
3.2	Dataset	12
3.3	Data Preprocessing	13
4.	DESIGN DETAILS	
4.1	Novelty	14
4.2	Innovativeness	14
4.3	Interoperability	14
4.4	Performance	14
4.5	Security	15
4.6	Reliability	15
4.7	Maintainability	15
4.8	Portability	15
4.9	Legacy to Modernization	16
4.10	Reusability	16
4.11	Application Compatibility	16
4.12	Resource Utilization	16
5.	HIGH LEVEL SYSTEM DESIGN /SYSTEM ARCHITECTURE	17
6.	DESIGN DESCRIPTION	
6.1	Master Class Diagram	19
6.2.	ER Diagram / Swimlane Diagram / State Diagram	20
6.3	User Interface Diagrams	21
6.4.	Report Layouts	22
6.5.	External Interfaces	23

1	6.6. Packaging and Deployment Diagram	24
7.	TECHNOLOGIES USED	25-26
8.	IMPLEMENTATION AND PSEUDOCODE	27-32
9.	CONCLUSION OF CAPSTONE PROJECT PHASE – 2	33
10.	PLAN OF WORK FOR CAPSTONE PROJECT PHASE - 3	34
4	REFERENCES/BIBLIOGRAPHY	37
	APPENDIX A DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	38

LIST OF TABLES

Table No.	Title	Page No.
1	Acronyms and Abbreviations	39

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	System Architecture Diagram	3 18
6.1	Class Diagram	19
6.2	ER Diagram	20
6.3	Use Case Diagram	21
6.4	External Interface Diagram	23
8.1	Face Mesh Detection	30
8.2	Yolo Detection	30
8.3	Alert Detection	31
8.4	Normal Driving Detection	32

CHAPTER 1

INTRODUCTION

The issue of road accidents is one of the significant issues related to the contemporary transport sector. Majority of the accidents arise because of human-related errors – fatigue, distraction, among others. An individual mistake could lead to the failure of the entire trip, particularly for longer trips characterized by busy traffic conditions. That is why efforts should be made to develop a smart system that is able to monitor the driver at all times.

In this research work, we will present an algorithm as well as a functioning prototype that uses a laptop camera to detect driver distraction through the use of computer vision and machine learning technologies. For this purpose, we employ the MediaPipe Face Mesh model to extract eye and mouth landmarks, and YOLOv8 for detecting distractors.

Our uniqueness lies in integrating different behaviors and tracking their consistency to enhance accuracy of the detector. Alerts are generated upon identifying anomalies to prevent dangers. Our future plans include designing, testing, and validation of the system.

CHAPTER 2

PROBLEM DEFINITION

The safety of the drivers lies at the core of every transport network. The vast majority of accidents occur due to the development of unsafe driver behavior: fatigue, distractions. Such issues are spontaneous and difficult to be detected unless constant surveillance is carried out.

Presently, almost all solutions either involve expensive hardware or detect only one particular type of dangerous behavior, making their performance impractical.

An efficient, affordable, real-time monitoring method involving detection of multiple types of unsafe behavior is required. It should use low-cost technology, provide accurate detection and deliver instant notifications about anomalies observed.

In our project, we develop a deep learning based technique that allows for instant detection of abnormal driver behavior using common webcams.

CHAPTER 3

DATA

3.1 Overview :

The model utilizes a combination of freely available datasets and data collected by us to make sure that the detection of driver behavior is efficient and reliable. Due to the combination of sources, the model becomes resilient in dealing with different real-life situations.

Most of the data is represented through images and videos showing drivers who demonstrate normal behavior, yawn, and display other symptoms of fatigue. The collection of such data is required to train the system and detect anomalies accurately.

By using both already existing datasets and input from real webcams, the model's performance under different circumstances is improved significantly.

On balance, the dataset is comprehensive enough to cover all the cases related to driver states and detect any anomalies.

3.2 Dataset :

The primary dataset that was utilized in this process is called the Yawning Detection Dataset (YawDD), which is concerned with detecting drowsiness through yawning. This dataset contains pictures and videos of people yawning while driving, yet it is quite limited since it deals only with one class.

To resolve this issue, a custom dataset was created using a webcam to introduce several classes into the system: normal driving, drowsiness, and yawning.

The data collection was carried out by taking video frames and marking them accordingly, creating a multi-class dataset more reflective of driving scenarios.

With the help of the YawDD dataset and the collected data, the efficiency of the system increased.

3.3 Data Preprocessing :

It is important to preprocess the information received from the camera before feeding it into the models. In this case, the initial footage is divided into single frames for further processing. After that, all the frames are resized in such a way to have the same resolution in terms of pixels.

The second step is the change of the color scheme of images from BGR to RGB. This will help to unify the images so that it could easily work with MediaPipe Face Mesh and YOLOv8.

Preprocessing helps to remove the inaccuracies in data, and increases the accuracy of the whole system, making it suitable for real-time usage.

Summing up, it is necessary to note that preprocessing makes the input data model-friendly and removes any inaccuracies.

CHAPTER 4

DESIGN DETAILS

4.1 Novelty :

This detection system is unique in the way that it incorporates several different methods into its detection process. Specifically, it utilizes facial landmarks along with the MediaPipe Face Mesh, as well as object detection by YOLOv8 to detect both physiological and behavioral anomalies. This system differs from its predecessors in the sense that unlike those, it provides a more comprehensive view of the behavior of a driver. Furthermore, it relies on ordinary webcam technology, which helps reduce expenses and ensures practicality. The inclusion of temporal detection further improves its detection capabilities.

4.2 Innovativeness :

It offers an innovative and multi-modal technique for tracking drivers through the use of several techniques of deep learning. As opposed to relying on a single model, the process takes into account the use of facial features alongside objects to make the process more reliable. Temporal analysis is used to track behaviors through time, as opposed to doing so based on a single frame to prevent false alerts, such as blinking. Emphasis has been put on practical implementation, as opposed to theoretical aspects

4.3 Interoperability :

The overall process is executed by the integration of a number of technologies including OpenCV, MediaPipe, and PyTorch in one chain of operations related to video processing. In their turn, each of the used technologies serves particular functions: OpenCV processes capturing and primary preprocessing of the video, MediaPipe, and YOLOv8 extract features from the frames. The information obtained with the help of these technologies is then united and delivered to further processing. Thus, internal interoperability of the used technologies guarantees their effective collaboration and future extensibility.

4.4 Performance :

This system is optimized for fast operation in real time, which allows it to detect objects very quickly. Through optimizing the size of the frame and effective pre-processing operations, the computations required are reduced. Lightweight algorithms such as those of MediaPipe Face Mesh and YOLOv8 enable maintaining quick processing operations. It is able to process many frames per second on an average laptop. Temporal logic ensures accurate detections without false alarms.

4.5 Security :

The system ensures security through the fact that it performs all operations internally on the device. Video clips are never sent to any server or cloud storage. This reduces the risk of data leakage or access by any unauthorized entity. Since the system uses a webcam, it is imperative to protect privacy, which becomes easier with local processing.

4.6 Reliability :

The system operates consistently in normal operations. The system can detect driver conditions such as sleepiness and distraction accurately in real-time. By integrating various forms of detection techniques, reliability is improved. Time-series analysis assists in reducing false alerts caused by short and non-essential transitions. However, high levels of illumination and visual obstructions may have an impact on performance. Overall, the system performs consistently under most conditions.

4.7 Maintainability :

It should be noted that the system uses the modular architecture that guarantees its maintenance and update ability. The preprocessing stage, feature extraction, and alerts creation operate separately. Therefore, any change within one of these components will not have an effect on other parts. This will simplify the debugging process because of separate operations. Moreover, when developing future iterations, there would be no need to rewrite the whole system.

4.8 Portability :

The solution is designed to be portable and will function on various platforms requiring minor modifications only. It will work on regular laptops, but it may also be optimized for embedded platforms. Using popular programming languages such as Python, along with popular libraries such as OpenCV, the algorithm will operate on all operating systems. The implementation does not require any specific hardware, which is an advantage for practical uses.

4.9 Legacy to Modernization :

The conventional approach to monitoring drivers relies heavily on the use of conventional sensors and heuristic techniques that may not always work effectively. The innovative approach replaces such conventional techniques by adopting a new approach based on the use of artificial intelligence. Thanks to computer vision, the technique accurately monitors driver behavior. Compared to conventional systems, this new one has better efficiency and flexibility and is capable of producing more effective outcomes.

4.10 Reusability :

The elements built for this system are not static and can be used in other systems as well. Consider the facial recognition or object recognition components; they are flexible enough to be used in a surveillance system or an intelligent monitoring system. As the system is modular, removing a single element will be relatively easy, reducing development time in future endeavors.

4.11 Application Compatibility :

This solution can fit into various software ecosystems and can even be connected to other software applications. The system can be scaled up to interface with intelligent transportation systems or safety systems. Being dependent on generic programming techniques makes it possible to work in several different environments. In the future, it is also possible to integrate the solution into mobile and embedded software applications.

4.12 Resource Utilization :

This system is designed in such a way that it makes maximum use of its available resources. It makes use of light models in order to reduce the workload on the CPU and minimize memory usage. Unnecessary information is stripped by preprocessing and therefore increases its performance. The system works perfectly on a regular laptop without requiring high-end equipment.

CHAPTER 5

SYSTEM ARCHITECTURE

This system employs a modular architecture, with various stages in the pipeline designed to process driver data to detect odd behaviors. As each stage takes care of a particular process, the pipeline remains efficient and structured.

First, the input stage takes video footage in real-time from a webcam and divides it into several frames for further processing.

Preprocessing involves resizing each frame and converting them from BGR to RGB to ensure compatibility with the deep learning algorithms.

There are two parts involved in feature extraction, including the media pipe face mesh algorithm for facial landmark detection, which identifies features like eyes and mouths, and YOLOv8 for object recognition (mobile devices) used as indications of distracting behavior.

Behavior analysis is done using temporal logic to distinguish between normal and anomaly patterns in the driver's actions.

The last stage involves classification and alerts that notify whether a driver is distracted or not through either audio or visual alerts.

In summary, this approach makes it easy for scalability and maintenance.

ARCHITECTURE DIAGRAM :

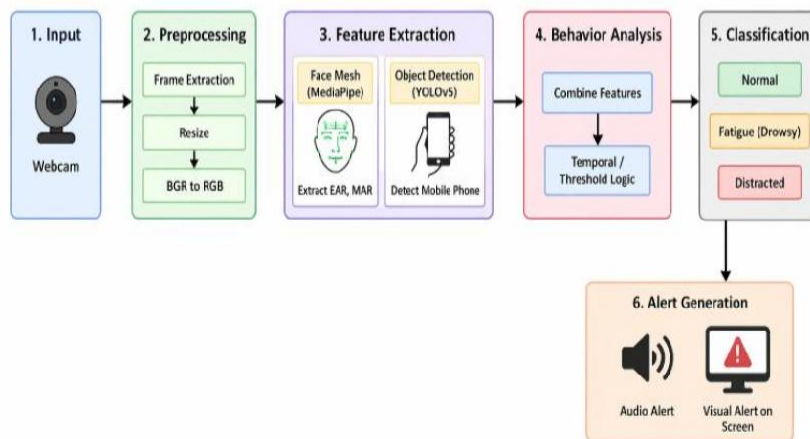
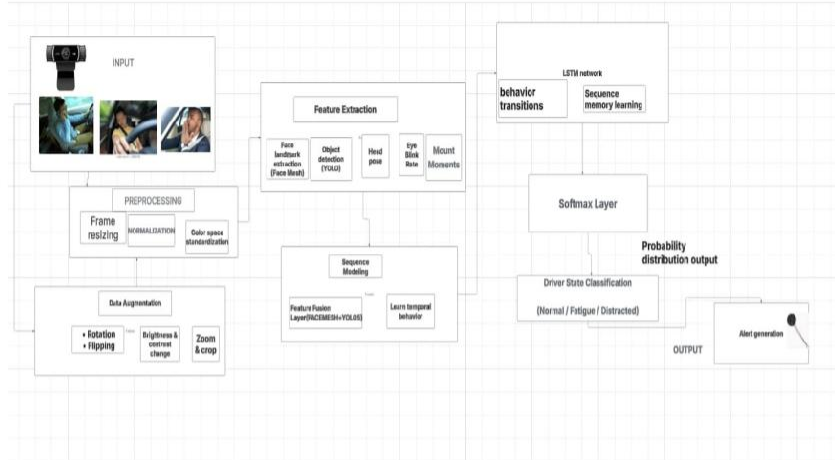


Figure 5.1

CHAPTER 6

DESIGN DESCRIPTION

6.1 Master Class Diagram

Master class diagram depicts the structure of the system, showing all its modules and their interactions. The video input is processed by the VideoCaptureManager module, and the facial data is managed by the FaceDetector and FaceMeshExtractor modules. The behaviour patterns are analyzed by the BehaviourAnalyzer module, and the drowsiness detection is performed by the DrowsinessDetector module. Alerts are generated by the AlertManager module, and event data is logged by the DataLogger module.

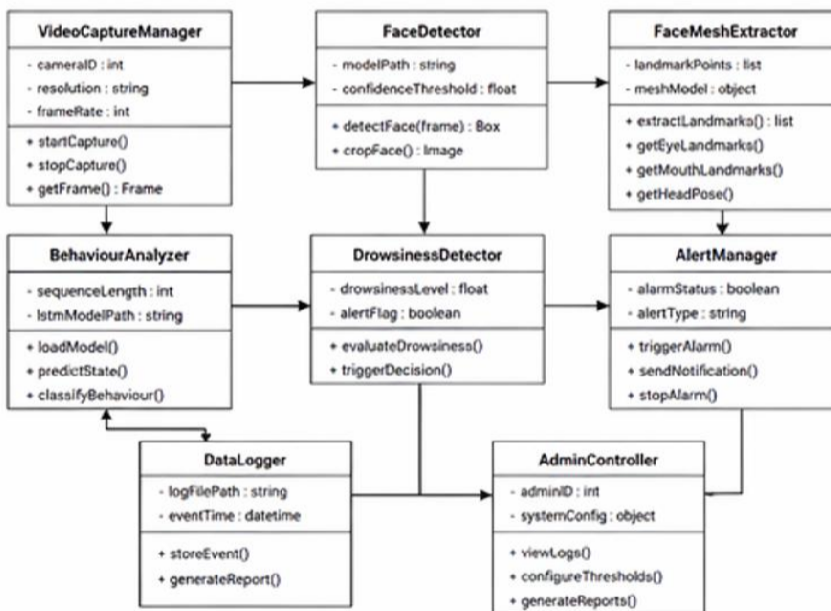


Figure 6.1

6.2 ER DIAGRAM

The ER diagram represents how the database of the system should be logically organized. One driver might participate in more than one session, but while conducting each session, the system generates detections that might detect any form of drowsiness or distraction. Whenever such a detection is made, alerts are generated for the user's attention. All events get logged into the system logs, which are monitored by the admin account.

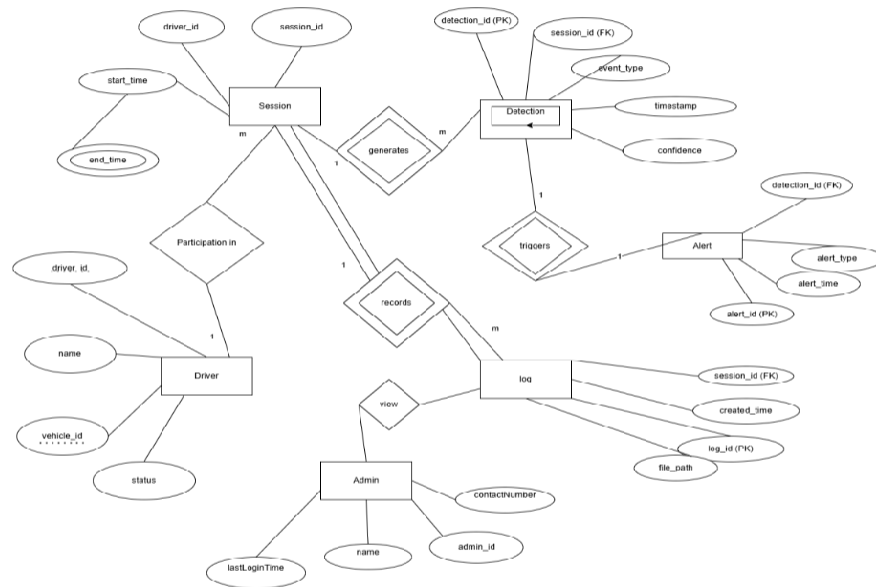


Figure 6.2

6.3 User Interface Diagrams

Figure represents the interaction between users and the system functionalities. The primary roles include Driver and Admin. The Driver has the power to control the monitoring process and turn it on and off, whereas the system captures the video, analyzes the facial features, tracks landmarks, and detects anomalies like drowsiness and distraction. Upon detection of any anomalies, the system sends out alerts and notifications. On the other hand, the Admin has the role of viewing the log files, setting thresholds, and producing reports.

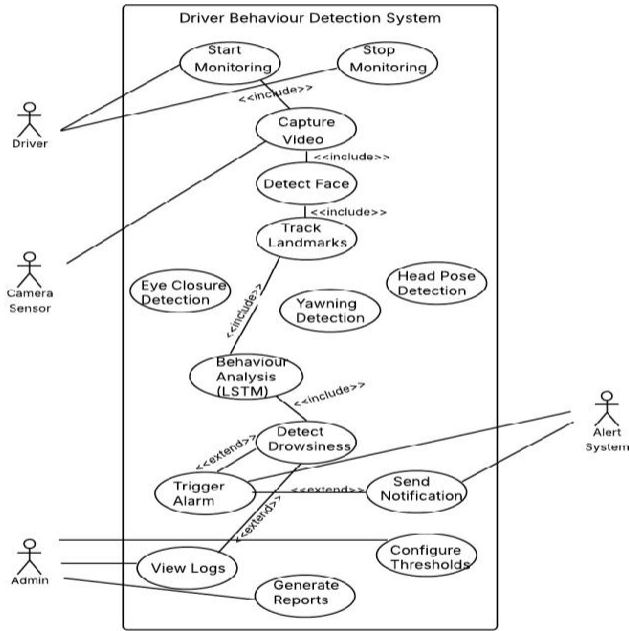


Figure 6.3

6.4 Report Layouts

The report layout reveals how the system stores and displays information regarding the driver behaviors that it detects. The system generates useful and informative logs and reports containing any driver behavior anomalies such as sleepiness and distractions. Each event contains important information including the exact timing, type of anomaly, confidence level and if there were alerts raised.

The reports will be designed to make the data easy and understandable for quick review by the user. This data will be saved in the form of a log file or in a database from which the system administrator can easily retrieve it.

Furthermore, the system can generate real-time reports in the display area including any overlay information and alerts. In all, the reports are used in tracking the driver behaviors and identifying patterns of unsafe driver behaviors.

6.5 External Interfaces

Figure illustrates the system's connection with entities from the outside world. Video data is supplied by a camera, and the driver symbolizes the person under surveillance. Model development and enhancements are facilitated by a training set. The administrator employs the system to make adjustments to its configurations and monitor log files. Alerts are delivered using an alerting system and a notification service. In addition, all incidents are logged in a data logger.

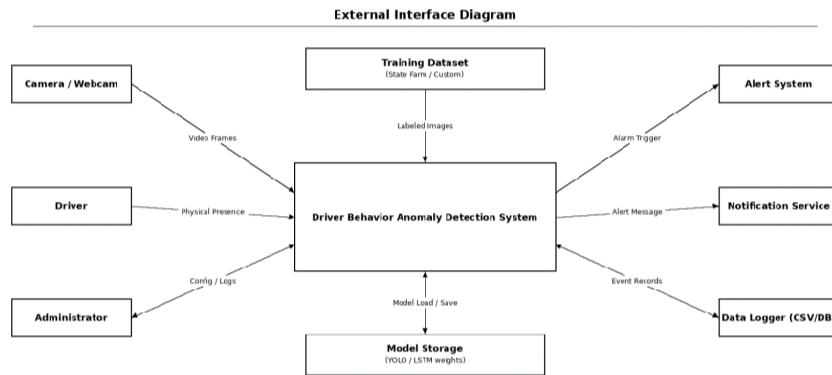


Figure 6.4

6.6 Packaging and Deployment Diagram :

The packaging scheme determines the structure of the system's software modules and their execution flow. The proposed anomaly detection system for drivers' behavior is developed using the Python language and divided into the following modules: video capture, preprocessing, feature extraction, behavior analysis, and alert generation. Each module is implemented in separate files for modularity, readability, and ease of maintenance.

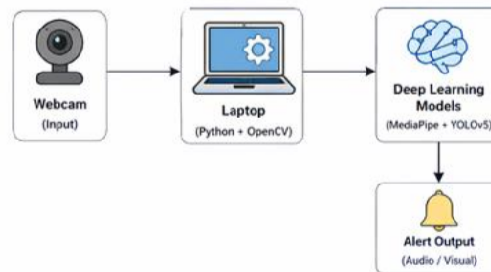
Deep learning libraries such as MediaPipe Face Mesh and YOLOv8 are included in the package as external modules and models. Configurable parameters that include detection threshold values and conditions under which alerts are generated are used as configurable settings. The system stores logs containing the results of anomaly detection and alerts generated by the system.

The deployment diagram illustrates how the physical components of the system will be assembled. A webcam captures real-time images and sends them to the laptop where the application is installed. The captured images are analyzed using Deep Learning technologies such as MediaPipe Face Mesh which maps the facial landmarks and YOLOv8 used to detect objects in the video stream.

Based on these features, the system can determine any irregular conditions including the level of drowsiness and distraction. When an anomaly is detected, the system generates alerts either in sound form or visually by displaying messages. Events generated by the system are stored for future analysis. The emphasis in this diagram is on the interplay between hardware and software elements in real-time processing.

Packaging Structure:

```
project/
├── main.py
├── preprocessing.py
├── face_mesh.py
├── yolo_detection.py
├── behavior_analysis.py
├── alert.py
├── models/
└── logs/
```



CHAPTER 7:

TECHNOLOGIES USED

This research project involves integrating various kinds of software tools for constructing an online driver behavior anomaly detection system. The selected technology emphasizes quick and efficient video analysis capabilities and high-performing features for the system. With the use of computer vision and deep learning, it is possible to detect behavior anomalies such as drowsiness and distraction.

7.1 Python :

The programming language used for the system is Python because of its simplicity and versatility. It establishes an ideal ecosystem to develop Deep Learning and Computer Vision algorithms, and it is compatible with various libraries such as OpenCV, MediaPipe, and PyTorch.

7.2 OpenCV :

OpenCV deals with the capture and manipulation of videos and images. It captures videos from the camera source and converts the stream to single frames. In addition, it also includes processes such as resizing and conversion between color spaces. Moreover, it also displays these processed frames.

7.3 MediaPipe Face Mesh :

MediaPipe Face Mesh acts as a facial landmark detector that detects the detailed landmarks of various parts of the face like eyes, mouth, and head pose using 468 landmarks. The EAR and MAR ratios are derived from the calculated landmarks. These ratios are used for detecting drowsiness and yawning.

7.4 YOLOv8 :

YOLOv8, which stands for You Only Look Once, is used to detect objects in real time. YOLOv8 is ideal for detecting objects that can be distractions, such as mobile phones.

7.5 PyTorch :

PyTorch is an open-source deep learning library, which acts as a framework for developing and deploying models. PyTorch provides a set of tools for developing neural networks and doing other tasks related to models. For example, YOLOv8 is deployed using PyTorch.

CHAPTER 8 :

IMPLEMENTATION AND PSEUDOCODE

8.1 Implementation :

This model consists of the combination of programming through Python along with computer vision techniques, including deep learning methods. Video streaming data is captured from the laptop camera and processed frame-by-frame where frame-wise preprocessing is performed through scaling and converting color spaces.

MediaPipe Face Mesh is employed for landmark detection and obtaining coordinates of important facial points, such as the locations of the eye and the mouth. Further, the values of Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) are calculated as signs of drowsiness and sleepiness.

To detect distracting items, YOLOv8 is applied to identify objects in the picture and determine their nature to distinguish smartphones as the source of distraction. Finally, results obtained via Face Mesh and YOLOv8 models are combined into one for analysis of the driver's behavior.

Using temporal logic for monitoring allows identifying the abnormal behavior only if it is observed during a certain time interval. Thus, warnings about drowsiness and distraction will be issued by the system.

8.2 System Workflow :

This approach works in an orderly sequence starting from video capture and ending with alert creation. The web camera captures real-time video that gets segmented into video frames. These frames go through the preprocessing phase before being subjected to the feature extraction phases. Classification of driver behavior occurs by combining facial and object information through temporal reasoning. Depending on the results, alerts get generated for display to the user.

8.3 Pseudocode :

```
Start system

Initialize webcam

While webcam is ON:
    Capture frame
    Resize frame
    Convert BGR to RGB

    Detect face landmarks using Face Mesh
    Calculate EAR and MAR

    Detect objects using YOLOv8

    Combine features (EAR, MAR, object detection)

    Apply temporal logic:
        If EAR < threshold for few seconds:
            Mark as Drowsy
        If phone detected continuously:
            Mark as Distracted

    If anomaly detected:
        Generate alert (audio/visual)

Display output frame

End system
```

8.4 Key Features Implemented :

This system provides real-time video analysis including facial feature detection, object recognition, and anomaly classification, while also producing alert notifications and maintaining comprehensive logs.

The modular design of this system allows for easy addition of more features in the future.

8.5 Eye Aspect Ratio (EAR) :

The Eye Aspect Ratio (EAR) is used to detect eye closure and drowsiness. It is calculated using distances between eye landmarks.

$$EAR = (||p2 - p6|| + ||p3 - p5||) / (2 * ||p1 - p4||) \quad (8.1)$$

Where p1 to p6 represent eye landmark points. A lower EAR value indicates closed eyes.

8.6 Mouth Aspect Ratio (MAR) :

The Mouth Aspect Ratio (MAR) is used to detect yawning.

$$MAR = (||p3 - p7|| + ||p4 - p6||) / (2 * ||p1 - p5||) \quad (8.2)$$

A higher MAR value indicates mouth opening, which may suggest yawning.

8.7 Implementation Results / Output Screenshots :

The images below depict how the anomaly detection system for the driver behavior works in real-time. They include the process of detecting the landmarks of the face, objects, and issuing alerts.

1. Face Mesh Detection :



Figure 8.1 : Facial landmark detection using MediaPipe Face Mesh

2. YOLO Detection :



Figure 8.2 : Object detection (mobile phone) using YOLOv8

3. Alert Detection :



Figure 8.3 : Alert generated during drowsiness/distracted detection

4. Normal Driving :



Figure 8.4

CHAPTER 9 :

CONCLUSION OF CAPSTONE PROJECT PHASE - 2

For Phase 2, our efforts were focused on constructing a prototype of the driver behavior anomaly detection system based on the theoretical design created in Phase 1. We constructed a system that monitors driver behavior in real-time using a webcam attached to a laptop.

It makes use of the MediaPipe Face Mesh technology to capture face landmark data and the YOLOv8 algorithm for object detection to detect signs of distraction, such as the use of a mobile phone. The video data is analyzed by extracting relevant features to identify anomalies related to drowsiness and distraction.

This was verified in real-time settings, proving its capability to detect dangerous driving behaviors. Its modularity makes it highly flexible and scalable.

As such, we gained practical experience in the application of deep learning and computer vision technologies through Phase 2. In addition, this paves the way for Phase 3 in which improvements will be made.

Beyond implementation, performance tests were done to determine how effective the system is and its reliability under varying conditions. From the results, it is clear that this system can effectively detect different states that a driver may have based on their appearance and objects within the car. Temporal logic has helped in reducing false detection as a condition must be present for a defined period before it is recorded.

The system operates effectively under normal circumstances, but there are certain limitations which include sensitivity to light levels and partial coverage of the driver's face by an object. These are key concerns which would need more improvement to make the system more effective and efficient. The overall result is that a viable solution to the real-time monitoring of drivers has been achieved using low-cost technology.

In conclusion, the Phase 2 tests confirm that the proposed system is feasible and gives a good platform for the next phase of enhancements during Phase 3.

CHAPTER 10 :

PLAN OF WORK FOR CAPSTONE PROJECT PHASE - 3

10.1 Overview :

The third **phase** centers on developing and improving the driver behavior anomaly detection algorithm that was created during the second phase. The goal here is to improve its accuracy, increase performance, and make it more robust to be used in practice. This phase will involve the development of more sophisticated model integration and overall system optimization.

10.2 Planned Work :

10.2.1 Model Improvement :

The present system shall undergo an upgrade in order to enhance the accuracy of detection. This encompasses optimization of YOLOv8 and tweaking the thresholds for facial features such as EAR and MAR. Generalization can be enhanced by incorporating additional training data.

10.2.2 Temporal Modeling Enhancement :

Temporal analysis techniques that can be applied include Long Short Term Memory (LSTM). The use of such analysis techniques will improve the ability of the algorithm to distinguish between normal and anomalous behavior over time.

10.2.3 System Optimization :

This will ensure that the system operates faster and without any delays. Various ways in which we could achieve this include downsizing of the system, frame-skipping, and optimal resource management.

10.2.4 Dataset Expansion :

Further datasets or user inputs will be used to enhance the reliability of the model. This process requires obtaining videos of driving in real-time under varying situations such as lighting, angles, and driving actions.

10.2.5 Alert System Enhancement :

The alarm will be improved by incorporating different levels according to the severity of the scenario. To enable the driver to have a better indication, we could incorporate sound, visual, and even vibrations.

10.2.6 Testing and Evaluation :

The tests will involve live input, and we shall observe the performance of the system through various tests that will measure the accuracy, precision, recall, and speed of the system. Moreover, the system will be tested in various environments.

10.2.7 Deployment Enhancement :

The platform will also be fine-tuned in order for it to operate efficiently on both embedded systems and mobile devices.

10.2.8 Documentation and Final Report :

Finally, the completion of the system documentation, including all information related to the design, implementation, results, and performance analysis, is involved. We will complete the project report and presentation.

REFERENCES :

- [1] M. A. Uddin et al., “Abnormal Driving Behavior Detection: A Machine and Deep Learning Based Hybrid Model,” *Int. J. Intell. Transp. Syst. Res.*, 2025.
- [2] E. Civik and U. Yuzgec, “Real-Time Driver Fatigue Detection System with Deep Learning on a Low-Cost Embedded System,” *Microprocessors Microsyst.*, 2023.
- [3] F. Qu et al., “Comprehensive Study of Driver Behavior Monitoring Systems Using Computer Vision and Machine Learning Techniques,” *J. Big Data*, 2024.
- [4] B. B. Gupta et al., “Deep Learning Model for Driver Behavior Detection in Cyber-Physical System-Based Intelligent Transport Systems,” *IEEE Access*, 2024.
- [5] S. Abbas et al., “Naturalistic Driving Data-Based Anomalous Driving Behavior Detection Using Hypertuned Deep Autoencoders,” *Electronics*, 2023.
- [6] A. Author et al., “Enhanced Deep Neural Network Model for Anomalous Driving Behavior Detection,” *Wiley J. Surveillance*, 2024.
- [7] R. Kumar and S. Gupta, “Driver Drowsiness Detection Using CNN-LSTM Hybrid Deep Learning Model,” *IEEE Sensors J.*, 2023.
- [8] H. Zhang et al., “Multi-Modal Driver Fatigue Detection Using Infrared and RGB Fusion,” *IEEE Trans. Intell. Veh.*, 2023.

APPENDIX A: DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

This appendix serves as an explanation of the most important terminology, acronyms, and abbreviations that have been used in this project. This appendix aims at helping readers understand the professional language used in this report.

Definitions :

Drowsiness Detection: The process of identifying whether a driver is fatigued or sleepy using facial features such as eye closure.

Distraction Detection: The process of identifying driver activities such as mobile phone usage that divert attention from driving.

Feature Extraction: The process of extracting meaningful information from input data, such as facial landmarks from images.

Temporal Analysis: The process of analyzing data over time to detect patterns or anomalies.

Real-Time System: A system that processes data and produces output instantly with minimal delay.

Acronyms and Abbreviations :

Term	Full Form	Description
AI	Artificial Intelligence	Simulation of human intelligence using machines
DL	Deep learning	Subset of AI using neural networks
CNN	Convolutional Neural Network	Model used for image processing
YOLO	You Only Look Once	Real-time object detection algorithm
EAR	Eye Aspect Ratio	Measure used to detect eye closure
MAR	Mouth Aspect Ratio	Measure used to detect yawning
FPS	Frames Per Second	Number of frames processed per second
RGB	Red Green Blue	Color format used in deep learning
BGR	Blue Green Red	Color format used in OpenCV
API	Application Programming Interface	Interface for communication between software components

APPENDIX B: USER MANUAL :

This section provides instructions for using the driver behaviour anomaly detection system. It explains how to run the system and interpret the outputs.

System Requirements :

- Laptop or PC
- Webcam
- Python installed
- Required libraries:
 - OpenCV
 - MediaPipe
 - PyTorch

Steps to Run the System :

1. Open the project folder
2. Run the main Python file:
 - a. **Python main.py**
3. The webcam will start automatically
4. The system begins monitoring driver behavior

System Features :

- Real-time video processing
- Face landmark detection
- Object detection (mobile phone)
- Drowsiness detection
- Alert generation

Output Description :

- Detection results are shown on the screen
- Alerts are displayed when anomalies are detected
- Logs are stored for future reference

Precautions :

- Ensure proper lighting conditions
- Keep face visible to the camera
- Avoid camera obstruction

Capstone_Report_Team_8

ORIGINALITY REPORT

11%	2%	2%	9%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to PESU (Group) Student Paper	7%
2	B.G.D.A. Madhusanka, Sureswaran Ramadass, Premkumar Rajagopal, H.M.K.K.M.B. Herath. "Artificial Intelligence–Based System for Gaze–Based Communication", CRC Press, 2024 Publication	1%
3	Pushpa Choudhary, Sambit Satpathy, Arvind Dagur, Dharendra Kumar Shukla. "Recent Trends in Intelligent Computing and Communication", CRC Press, 2025 Publication	< 1%
4	Submitted to PES University Student Paper	< 1%
5	Submitted to CSU, San Jose State University Student Paper	< 1%
6	M. Adil Khan, Tahir Nawaz, Umar S. Khan, Amir Hamza, Nasir Rashid. "IoT–based Non–intrusive Automated Driver Drowsiness Monitoring Framework for Logistics and Public Transport Applications to Enhance Road Safety", IEEE Access, 2023 Publication	< 1%
7	Submitted to Bahrain Polytechnic Student Paper	< 1%
8	Submitted to University of Hertfordshire Student Paper	< 1%
9	Submitted to De Montfort University Student Paper	< 1%

10	rctce2026.tce.edu Internet Source	< 1 %
11	www.mdpi.com Internet Source	< 1 %
12	journalofbigdata.springeropen.com Internet Source	< 1 %
13	Tresa Maria Josylin, Ganeshayya Shidaganti, Vishwachetan Dasegowda, Anasuya Jadagerimath, Prakash Sheelvanthmath. "A multimodal framework for secure digital document fortification using Morse code and biometric watermarking", Bulletin of Electrical Engineering and Informatics, 2026 Publication	< 1 %

Exclude quotes On
Exclude bibliography On

Exclude matches Off